

1. Принцип ACID в контексте реляционных СУБД ACID — это фундаментальный набор требований к транзакциям, который гарантирует сохранность данных даже в случае сбоев.

- Atomicity (Атомарность): Гарантирует, что транзакция будет выполнена полностью или не будет выполнена совсем. Если хоть одна операция внутри транзакции даст сбой, все предыдущие изменения откатываются (ROLLBACK).
- Consistency (Согласованность): Перед началом и после завершения транзакции база данных должна находиться в согласованном состоянии. Это значит, что все правила (ограничения, ключи, триггеры) соблюдены.
- Isolation (Изоляция): Параллельные транзакции не должны влиять друг на друга. Результат выполнения нескольких транзакций одновременно должен быть таким же, как если бы они выполнялись по очереди.
- Durability (Долговечность): Если пользователь получил подтверждение о завершении транзакции (COMMIT), изменения должны быть надежно сохранены (обычно в лог на диске), и никакие сбои питания не должны их стереть.

2. Отличие реляционной модели от документной (NoSQL)

- Реляционная модель (SQL): Данные хранятся в жестко структурированных таблицах. Связи между ними строятся через внешние ключи. Требуется предварительного описания схемы (DDL).
 - Плюсы: Строгий контроль типов, поддержка сложных SQL-запросов, транзакции ACID.
 - Пример: PostgreSQL, MySQL.
- Документная модель (NoSQL): Данные хранятся в виде гибких документов (обычно JSON или BSON). Схема может меняться на лету (Schemaless) — в разных записях одной коллекции могут быть разные поля.
 - Плюсы: Легкое горизонтальное масштабирование (шардинг), высокая скорость работы с иерархическими данными.
 - Пример: MongoDB.

3. Нормализация БД и нормальные формы Нормализация — это процесс проектирования БД, направленный на устранение дублирования данных и исключение аномалий (ошибок при обновлении, удалении или вставке).

- 1НФ (Первая нормальная форма): Все значения в ячейках атомарны (нельзя хранить список телефонов в одной строке), и нет повторяющихся полей.
- 2НФ: Соблюдена 1НФ + все неключевые поля зависят от полного первичного ключа (важно для составных ключей). Поля, зависящие только от части ключа, выносятся в отдельные таблицы.
- 3НФ: Соблюдена 2НФ + нет транзитивных зависимостей. Это значит, что неключевые поля должны зависеть только от первичного ключа, а не от других неключевых полей (например, «Город» зависит от «ID магазина», а не от «ID сотрудника»).

4. Типы JOIN в SQL Операторы JOIN используются для объединения строк из двух или более таблиц на основе общего условия.

- INNER JOIN: Возвращает только те строки, для которых есть совпадения в обеих таблицах.
- LEFT (OUTER) JOIN: Возвращает все строки из левой таблицы и совпавшие строки из правой. Если совпадения нет, подставляются NULL.
- RIGHT (OUTER) JOIN: Зеркален левому — все строки из правой и совпадения из левой.
- FULL (OUTER) JOIN: Возвращает все строки из обеих таблиц. Если пары нет — заполняет NULL.
- CROSS JOIN: Создает декартово произведение (каждая строка первой таблицы с каждой строкой второй).

```
SELECT orders.id, customers.name  
FROM orders  
INNER JOIN customers ON orders.customer_id = customers.id;
```

5. Индексы в БД и их виды Индекс — это отдельная структура данных (похожая на оглавление книги), которая ускоряет поиск данных за счет замедления

операций записи (INSERT/UPDATE). Виды:

- B-Tree (Сбалансированное дерево): Самый частый вид. Подходит для поиска точных значений и диапазонов (>, <, BETWEEN).
- Hash-индекс: Эффективен только для поиска по точному равенству (=).
- Кластерный индекс: Физически перестраивает порядок хранения строк в таблице (в таблице может быть только один такой индекс, обычно по первичному ключу).
- Некластерный индекс: Хранится отдельно от данных и содержит ссылки на физические строки.
- Составной индекс: Индекс по нескольким колонкам сразу (важен порядок колонок).

6. Elasticsearch и его эффективность Elasticsearch — это поисковая система на базе библиотеки Lucene, использующая инвертированный индекс (словарь слов со списком документов, где они встречаются). Сценарии эффективности:

- Полнотекстовый поиск: Когда нужно искать по части слова, учитывать морфологию (корень слова), синонимы и опечатки.
- Аналитика логов: Быстрый поиск по огромным объемам неструктурированных данных (стек ELK: Elasticsearch, Logstash, Kibana).
- Релевантность: ES умеет ранжировать результаты («этот документ подходит на 95%»). Реляционные СУБД на запросах LIKE '%текст%' работают крайне медленно, так как не могут использовать обычные индексы.

7. Репликация в СУБД Репликация — это механизм копирования данных с одного сервера БД (Master/Primary) на другие (Slave/Replica). Зачем нужна:

- Отказоустойчивость: Если «мастер» выйдет из строя, одна из «реплик» может стать новым мастером.
- Масштабируемость чтения: Все тяжелые запросы SELECT можно отправлять на реплики, разгружая основной сервер.
- Географическое распределение: Хранение копии данных ближе к пользователю для уменьшения задержки. Методы:
- Синхронная: Мастер ждет подтверждения от реплики о записи данных (медленно,

но надежно).

- Асинхронная: Мастер пишет данные у себя и сразу отвечает пользователю, отправляя копию на реплику позже (быстро, но есть риск потери данных при сбое).

8. Что такое секционирование таблиц? Приведите примеры.

Секционирование (Partitioning) — это метод разделения одной большой таблицы на более мелкие, физически независимые части, называемые секциями. При этом на логическом уровне (для пользователя и приложений) таблица остается единым объектом, и запросы к ней пишутся как обычно.

Основные цели:

- Производительность: СУБД использует технику «исключения секций» (Partition Pruning) — просматривает только те части данных, которые попадают под условия запроса, не сканируя всю таблицу.
- Упрощение обслуживания: Можно быстро удалять устаревшие данные (просто удалив старую секцию вместо долгого DELETE) или делать бэкап отдельных частей.
- Ускорение загрузки: Данные можно загружать в разные секции параллельно.

Примеры секционирования:

1. По диапазону (Range): Самый частый случай — по датам.

-- Пример: таблица продаж, разделенная по годам

```
CREATE TABLE sales (id int, sale_date date, amount decimal)
```

```
PARTITION BY RANGE (sale_date);
```

```
CREATE TABLE sales_2023 PARTITION OF sales FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');
```

```
CREATE TABLE sales_2024 PARTITION OF sales FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

2. По списку (List): По конкретным значениям. Например, по регионам: секция

«Север», секция «Юг».

3. Хеш-секционирование (Hash): СУБД сама распределяет данные по фиксированному числу секций на основе хеш-функции от ключа. Это полезно для равномерного распределения нагрузки.

4. Объясните, что такое транзакция и зачем она нужна.

Транзакция — это логически завершенная единица работы с базой данных, которая состоит из одного или нескольких SQL-запросов. Транзакция переводит базу из одного согласованного состояния в другое.

Зачем она нужна: Главная цель — гарантировать, что данные не будут повреждены при сбоях системы или при одновременной работе нескольких пользователей.

Транзакция обязана следовать принципам ACID:

- Атомарность (Atomicity): Либо все команды транзакции выполнятся успешно, либо ни одна из них не применится (откат — ROLLBACK).
- Согласованность (Consistency): Данные после транзакции должны соответствовать всем правилам и ограничениям БД.
- Изоляция (Isolation): Параллельные транзакции не должны мешать друг другу.
- Долговечность (Durability): Если транзакция зафиксирована (COMMIT), результаты не пропадут даже при выключении питания.

Классический пример: Перевод денег со счета на счет.

1. Списать 100 рублей у клиента А.
2. Начислить 100 рублей клиенту Б. Если между шагом 1 и 2 произойдет сбой, без транзакции деньги у клиента А пропадут, а у Б не появятся. Транзакция гарантирует, что либо обе операции пройдут, либо ни одна.
3. Чем отличается кластеризованный индекс от некластеризованного?

Индекс ускоряет поиск данных, но делает это по-разному.

Кластеризованный индекс (Clustered Index):

- Физический порядок: Он определяет порядок, в котором данные физически хранятся на диске. Если индекс по ID, то строка с ID=1 будет лежать перед ID=2.
- Количество: Может быть только один на таблицу (нельзя разложить одну стопку бумаг по алфавиту и по дате одновременно).
- Данные: Листовые узлы индекса содержат сами данные (строки таблицы).
- Обычно: Создается автоматически для первичного ключа (Primary Key).

Некластеризованный индекс (Non-Clustered Index):

- Физический порядок: Не влияет на то, как лежат данные на диске. Данные в таблице могут быть разбросаны хаотично.
- Количество: На одну таблицу можно создать множество индексов (по почте, по фамилии и т.д.).
- Данные: Хранится отдельно от таблицы. Его листовые узлы содержат только индексируемое значение и указатель (ссылку) на реальную строку в таблице.
- Аналогия: Некластеризованный индекс — это алфавитный указатель в конце учебника, а кластеризованный — это сам текст учебника, организованный по главам.

11. Что такое представление? В каких типах БД встречаются представления и зачем они необходимы?

Представление (View) — это «виртуальная таблица», содержимое которой определяется SQL-запросом. В отличие от обычной таблицы, представление не хранит данные физически (в момент обращения к нему СУБД просто выполняет сохраненный запрос).

Где встречаются: Практически во всех современных реляционных СУБД (PostgreSQL,

MySQL, MS SQL Server, Oracle, SQLite).

Зачем они необходимы:

1. Безопасность: Можно дать пользователю доступ к View, где видны только ФИО сотрудника, но скрыть доступ к основной таблице, где лежат паспортные данные и зарплаты.
2. Упрощение сложных запросов: Если у вас есть запрос с пятью JOIN и сложными формулами, вы можете сохранить его как View и обращаться к нему простым `SELECT * FROM my_view`.
3. Независимость данных: Если структура исходных таблиц изменится (например, разделите одну таблицу на две), вы можете просто переписать запрос внутри View, и клиентские приложения, работающие с этим View, не сломаются.
4. Консистентность: Обеспечивает единую логику выборки данных для всех разработчиков.

-- Пример создания представления

```
CREATE VIEW active_users AS  
SELECT id, name, email  
FROM users  
WHERE status = 'active' AND last_login > '2023-01-01';
```

12. Перечислите этапы проектирования БД.

Проектирование базы данных — это итерационный процесс перехода от бизнес-требований к физической реализации.

1. Анализ требований: Сбор сведений о том, какие объекты нужно хранить, какие между ними связи и какие задачи будет решать система. Итог — техническое задание (ТЗ).
2. Концептуальное проектирование: Создание высокоуровневой схемы данных, не зависящей от конкретной СУБД. На этом этапе строится ER-диаграмма (сущности, атрибуты и связи).

3. Логическое проектирование: Преобразование концептуальной схемы в реляционную (табличную) модель. Здесь определяются типы данных, первичные и внешние ключи, и проводится нормализация (обычно до 3НФ) для устранения избыточности.
4. Физическое проектирование: Оптимизация модели под конкретную СУБД (Postgres, Oracle и т.д.). Выбираются стратегии индексирования, секционирования, настраиваются параметры хранения данных на диске.
5. Реализация и поддержка: Написание SQL-скриптов (DDL) для создания таблиц, заполнение данными и мониторинг производительности запросов.
6. Для чего используется ER-диаграмма?

ER-диаграмма (Entity-Relationship Diagram) — это визуальный инструмент для моделирования структуры данных. Она состоит из сущностей (прямоугольники), атрибутов (овалы) и связей (ромбы или линии).

Зачем она используется:

- Визуализация логики: Помогает разработчикам и бизнес-аналитикам «увидеть» структуру данных и убедиться, что все бизнес-требования учтены.
- Универсальный язык: Позволяет обсуждать схему данных без привязки к коду или конкретной СУБД.
- Фундамент для разработки: На основе ER-диаграммы автоматически или вручную генерируются таблицы базы данных.
- Документирование: Служит справочником для новых разработчиков, объясняя, как данные связаны между собой (например, что один «Заказ» всегда принадлежит одному «Клиенту»).

14. Что такое «медленный запрос»? Как его оптимизировать?

Медленный запрос — это SQL-запрос, время выполнения которого превышает

установленный порог (например, 1 секунду) или вызывает значительную нагрузку на ресурсы сервера (CPU, RAM, диск).

Способы оптимизации:

1. Использование индексов: Самый эффективный способ. Индекс позволяет СУБД находить строки без полного сканирования таблицы (Full Table Scan).

-- Было медленно (поиск по фамилии без индекса)
CREATE INDEX idx_last_name ON users(last_name); -- После создания станет быстро
2. Анализ через EXPLAIN: Использование команды EXPLAIN ANALYZE для просмотра плана выполнения запроса и поиска «узких мест».
3. Отказ от SELECT *: Следует выбирать только необходимые столбцы, чтобы уменьшить объем передаваемых данных.
4. Оптимизация JOIN: Убедиться, что столбцы, по которым происходит объединение таблиц, проиндексированы.
5. Пагинация: Вместо выгрузки всех 100 000 строк использовать LIMIT и OFFSET.
6. Денормализация: В крайне нагруженных системах иногда сознательно добавляют избыточные данные, чтобы избежать тяжелых объединений (JOIN).
7. Назовите преимущества и недостатки MongoDB.

MongoDB — это документоориентированная NoSQL СУБД, где данные хранятся в формате BSON (похожем на JSON).

Преимущества:

- Гибкая схема (Schemaless): В одну коллекцию можно записывать документы с разным набором полей. Это идеально для быстро меняющихся проектов и

прототипов.

- Горизонтальное масштабирование: Встроенная поддержка шардирования позволяет легко распределять данные между десятками серверов.
- Высокая скорость: Благодаря хранению связанных данных в одном документе (вложенные объекты), чтение часто происходит быстрее, чем в реляционных БД с их JOIN.
- Удобство для разработчиков: Формат документов напрямую мапится на объекты в коде (JS, Python).

Недостатки:

- Потребление памяти: MongoDB активно использует оперативную память для кэширования, что требует мощного железа.
- Отсутствие сложных JOIN: Хотя есть оператор \$lookup, он работает значительно медленнее и менее гибко, чем JOIN в SQL.
- Объем данных на диске: Из-за дублирования имен полей в каждом документе база занимает больше места, чем реляционная.
- Транзакционность: Хотя транзакции появились (с версии 4.0), они работают сложнее и медленнее, чем в классических реляционных СУБД.

16. Объясните, что такое шардирование данных.

Шардирование (Sharding) — это стратегия горизонтального масштабирования базы данных, при которой данные распределяются между несколькими независимыми серверами (шардами). В отличие от вертикального масштабирования (покупки более мощного сервера), шардирование позволяет распределить нагрузку на множество обычных машин.

Принцип работы: Выбирается ключ шардирования (например, `user_id`). На основе этого ключа СУБД решает, на какой сервер отправить данные.

- Шардирование по диапазонам: Пользователи с ID 1–1000 на Сервере 1, 1001–2000 на Сервере 2.
- Хеш-шардирование: Применяется хеш-функция к ID, результат которой определяет

номер сервера. Это обеспечивает более равномерное распределение.

Зачем нужно:

- Когда объем данных превышает емкость дисков одного сервера.
- Когда количество запросов в секунду превышает пропускную способность одного процессора или памяти.

17. Что такое NewSQL? Приведите примеры СУБД.

NewSQL — это класс современных реляционных СУБД, которые стремятся совместить масштабируемость NoSQL-систем (способность работать на сотнях серверов) с гарантиями ACID и мощностью языка SQL традиционных БД.

Особенности:

- Масштабируемость: Поддержка распределенной архитектуры «из коробки».
- Транзакционность: Полная поддержка ACID в распределенной среде.
- Интерфейс: Использование стандартного SQL.

Примеры СУБД:

- Google Spanner: Глобально распределенная база данных, использующая атомные часы для синхронизации.
- CockroachDB: Облачная БД, устойчивая к сбоям (может пережить падение целого дата-центра).
- VoltDB: СУБД, оптимизированная для обработки данных в оперативной памяти (In-memory).

18. Зачем нужны триггеры в БД?

Триггер — это специальный тип хранимой процедуры, который автоматически выполняется (срабатывает) при возникновении определенного события в базе данных (INSERT, UPDATE или DELETE).

Зачем они нужны:

1. Соблюдение бизнес-логики: Например, при удалении пользователя триггер может автоматически перенести его данные в таблицу архива.
2. Аудит и логирование: Запись истории изменений (кто, когда и какое поле изменил) в специальную таблицу логов.
3. Валидация данных: Дополнительная проверка значений, которую сложно реализовать простыми ограничениями (Constraints).
4. Синхронизация: Обновление связанных данных (например, пересчет суммы заказа при добавлении нового товара).

Пример (псевдокод SQL):

```
CREATE TRIGGER update_stock
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE products SET stock = stock - NEW.quantity WHERE id = NEW.product_id;
END;
```

19. Какие облачные СУБД вы знаете? Назовите их особенности.

Облачные СУБД (Database-as-a-Service, DBaaS) — это сервисы, где провайдер берет на себя установку, настройку, резервное копирование и масштабирование базы данных.

Примеры и особенности:

- Amazon RDS (Relational Database Service): Позволяет развернуть классические БД (PostgreSQL, MySQL, Oracle) в облаке AWS.
 - Особенность: Автоматические обновления и бэкапы.
- Google BigQuery: Облачное хранилище данных для аналитики (OLAP).
 - Особенность: Способность обрабатывать терабайты данных за секунды без

необходимости настраивать индексы (Serverless).

- Azure Cosmos DB: Мультимодельная база данных от Microsoft.
 - Особенность: Гарантирует задержку менее 10 мс в любой точке мира и поддерживает разные модели (документную, графовую, ключевую).
- MongoDB Atlas: Облачная версия NoSQL базы MongoDB.
 - Особенность: Легкое развертывание кластеров с шардированием в любом облачном провайдере.

20. Что такое вложенные запросы в SQL? Приведите примеры.

Вложенный запрос (подзапрос) — это запрос SELECT, который находится внутри другого SQL-запроса (главного запроса). Он используется для того, чтобы получить промежуточные данные, которые затем будут использованы главным запросом в качестве условия или данных для вывода.

Подзапросы могут находиться в операторах WHERE, HAVING, FROM и даже SELECT. Обычно внутренний запрос выполняется первым, а его результат передается во внешний запрос.

Пример 1: Вложенный запрос в WHERE (Самый частый случай) Задача: Найти всех сотрудников, чья зарплата больше средней зарплаты по компании.

```
SELECT name, salary
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Объяснение: Сначала база данных вычислит среднюю зарплату (подзапрос в скобках), допустим, получилось 50000. Затем главный запрос выведет всех, у кого зарплата больше 50000.

Пример 2: Использование с оператором IN Задача: Найти имена клиентов, которые делали заказы в 2024 году (данные в разных таблицах).

```
SELECT name
```

FROM Customers

WHERE id IN (SELECT customer_id FROM Orders WHERE year = 2024);

21. Что такое функции в SQL? Какие функции бывают? Приведите примеры.

Функция в SQL — это именованный блок кода, который принимает параметры (или работает без них), выполняет определенные действия и обязательно возвращает значение. Функции используются для вычислений, форматирования данных или работы со строками прямо в запросе.

Функции в SQL делятся на две главные категории (встроенные) и одну пользовательскую:

1. Агрегатные функции. Они работают с набором строк (колонкой) и возвращают одно итоговое значение.

- COUNT() — количество строк.
- SUM() — сумма значений.
- AVG() — среднее значение.
- MAX() / MIN() — максимальное/минимальное значение. Пример:

-- Считаем количество пользователей в базе

```
SELECT COUNT(*) FROM Users;
```

2. Скалярные функции. Они применяются к одному конкретному значению (к каждой строке индивидуально) и возвращают одно значение.

- Строковые: UPPER() (перевод в верхний регистр), LOWER(), LENGTH() (длина строки).
- Математические: ROUND() (округление), ABS() (модуль числа).

Пример:

-- Выводим имена в верхнем регистре (ИВАН, АННА)

```
SELECT UPPER(name) FROM Users;
```

3. Пользовательские функции. Это функции, которые программист пишет сам для специфичных задач бизнеса, если встроенных не хватает.

Ситуация: Допустим, в нашей базе данных есть таблица товаров с ценами. Нам нужно часто вычислять цену товара с учетом налога (например, НДС 20%). Чтобы каждый раз не писать формулу заново в каждом запросе, мы создадим свою функцию.

1. Создание пользовательской функции

Функция принимает цену (число), вычисляет налог и возвращает итоговую стоимость.

В MS SQL параметры обозначаются символом @, а перед телом функции пишется ключевое слово AS.

-- Создаем функцию с именем CalculateTax

```
CREATE FUNCTION dbo.CalculateTax
```

```
(
```

```
@price DECIMAL(10,2)
```

```
)
```

```
RETURNS DECIMAL(10,2) -- указываем, что функция вернет число с 2 знаками после запятой
```

```
AS
```

```
BEGIN
```

```
    -- Возвращаем цену + 20%
```

```
    RETURN @price + (@price * 0.20);
```

```
END;
```

2. Использование функции

Главное отличие функции от процедуры — функцию можно использовать прямо в обычном запросе SELECT, точно так же, как встроенные функции типа MAX() или UPPER().

Важное правило MS SQL: при вызове своей функции нужно

обязательно указывать схему, обычно это dbo.

-- Выводим название товара, обычную цену и цену с налогом

```
SELECT  
    product_name,  
    price,  
    dbo.CalculateTax(price) AS price_with_tax  
FROM Products;
```

22. Что такое процедура в SQL? Зачем нужны процедуры?

Хранимая процедура — это набор SQL-инструкций, который сохранен в базе данных под определенным именем. Вы можете вызвать эту процедуру в любой момент, чтобы выполнить весь сохраненный в ней код.

В отличие от функции, процедура не обязана возвращать значение. Она обычно используется для выполнения действий: изменения данных (INSERT, UPDATE, DELETE), создания таблиц.

Зачем нужны хранимые процедуры? (Преимущества):

1. Повторное использование кода: Написали сложную логику добавления заказа один раз, и вызываем ее откуда угодно.
2. Безопасность (Защита от SQL-инъекций): Приложению можно дать права только на запуск процедуры, не давая прямого доступа к таблицам базы данных.
3. Скорость работы: База данных заранее «понимает» код процедуры и запоминает самый быстрый путь для её выполнения.

Поэтому процедура работает быстрее обычного запроса (который серверу приходится читать и обдумывать каждый раз заново).

Пример создания и вызова процедуры:

В MS SQL параметры пишутся с символом @, перед телом процедуры нужно писать AS, а для вызова используется команда.

-- Создание процедуры для добавления нового сотрудника

```
CREATE PROCEDURE AddEmployee
```

```
(
```

```
@emp_name VARCHAR(50),
```

```
@emp_salary INT
```

```
)
```

```
AS
```

```
BEGIN
```

```
    INSERT INTO Employees (name, salary)
```

```
    VALUES (@emp_name, @emp_salary);
```

```
END;
```

-- Вызов процедуры (передаем имя и зарплату)

```
EXEC AddEmployee 'Иван Иванов', 60000;
```

в чем разница между функцией и процедурой:

- Функция обязана вернуть значение, процедура — нет.
- Функцию можно использовать внутри запроса (например: SELECT моя_функция(id) FROM таблица). Процедуру в SELECT использовать нельзя, она вызывается отдельной командой (CALL или EXEC).